INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA IPCA

PROJETO GESTÃO DE ANTENAS

ESTRUTURAS DE DADOS AVANÇADAS

Aluno

Filipe Alexandre Coelho Fernandes(a31498)

Professor

Luís Ferreira

abril de 2025

LISTA ABREVIATURAS E/OU SIGLAS

BFS – *Breadth-First Search*

DFS – *Depth-First Search*

EDA - Estrutura de Dados Avançadas

EST – Escola Superior de Tecnologia

IPCA – Instituto Politécnico do Cávado e do Ave

LESI - Licenciatura em Engenharia de Sistemas Informáticos

UC - Unidade Curricular

INDICE DE FIGURAS

Figura 1- Estruturas utilizadas..... 5
Figura 2 - Declaração das funções utilizadas no projeto 6
Figura 3 - Makefile..... 7
Figura 4 - Função para Criar Antena..... 8
Figura 5 - Função para Inserir Antenas 8
Figura 6 - Remover Antena 9
Figura 7 - Listar Antenas..... 9
Figura 8 - Procurar Antena 9
Figura 9 - Cálculo do número de interferências 10
Figura 10 - Exemplo cálculo Interferências.....11
Figura 11 - Calcular interferências 12
Figura 12 - CarregarAntenaDe Ficheiro 13
Figura 13 - GravarAntenasBinario 14
Figura 14 - Teste 1.....15
Figura 15 - Teste 2.....16
Figura 16 - Teste 3.....16
Figura 17 - Teste 4.....17
Figura 18 - Expressão da equação paramétrica da reta 24
Figura 19 - Interseção de dois segmentos de reta.....24
Figura 20 - Equação paramétrica do segmento AB 25
Figura 21 - Equação paramétrica do segmento CD.....25
Figura 22 - Igualdade entre as duas expressões.....25
Figura 23 - Aplicação das coordenadas dos pontos 25
Figura 24 - Organização das equações lineares.....25
Figura 25 - Forma matricial.....26
Figura 26 - Cálculo do determinante principal 27
Figura 27 - Cálculo do determinante de t.....27
Figura 28 - Cálculo do determinante de u 27
Figura 29 - Demonstração do cáculo de Dt e Du 27
Figura 30 - Cálculo da variável u.....27
Figura 31 - Cálculo da variável t 27
Figura 32 - Cálculos correspondentes às expressões matemáticas das equações paramétricas.....29
Figura 33 - Cálculo do determinante principal.....29
Figura 34 - Cálculo de Det(t), Det(u), e dos parâmetros t e u.....29
Figura 35 - Verificação dos parâmetros e cálculo das interseções 30
Figura 36 - DFS 30
Figura 37 - BFS 31

ÍNDICE

LISTA ABREVIATURAS E/OU SIGLAS i
INDICE DE FIGURAS ii
INTRODUÇÃO 1
1. FASE 1- LISTAS LIGADAS 2
1.1. ENQUADRAMENTO TEÓRICO 2
1.2. PROJETO DE GESTÃO DE ANTENAS 5
1.2.1 Objetivos do Projeto 5
1.2.2 Estrutura do Projeto 5
1.2.2.1 estruturas.h 5
1.2.2.2 FuncoesAntenas.h 6
1.2.2.3 FuncoesAntenas.c 6
1.2.2.4 main.c 6
1.2.2.5 Makefile 7
1.2.3 As Antenas 7
1.2.4 As Interferências 10
1.2.4.1 Cálculo do número de interferências 10
1.2.4.2 Posicionamento das interferências 10
1.2.4.3 Concretização do algoritmo para detetar interferências 12
1.2.5 Manipulação de ficheiros 13
1.3. RESULTADOS OBTIDOS 15
2. FASE 2 – GRAFOS 18
2.1. PROJETO DE GESTÃO DE ANTENAS 21
2.1.1 Objetivos do Projeto 21
2.1.2 Estrutura do Projeto 22
2.1.2.1 grafo_structs.h 22
2.1.2.2 grafo_funcoes.h/.c 22
2.1.2.3 – ficheiros.h/.c 23
2.1.2.4 – procura.h/.c 23
2.1.2.5 main.c 23

2.2 As Interseções 23
2.2.1 Concretização do algoritmo para detetar interseções 28
2.3 TRAVESSIA DE GRAFOS 30
REFERÊNCIAS BIBLIOGRÁFICAS 33
ANEXOS 34
ANEXO I – ENUNCIADO DO TRABALHO PRÁTICO (FASE 1) 35
ANEXO II – ENUNCIADO DO TRABALHO PRÁTICO (FASE 2) 39

INTRODUÇÃO

O presente relatório pretende dar resposta a um objetivo de avaliação da Unidade Curricular (UC): Estrutura de Dados Avançadas (EDA), inserida no plano de estudos da Licenciatura em Engenharia de Sistemas Informáticos (LESI), da Escola Superior de Tecnologia (EST), do Instituto Politécnico do Cávado e do Ave (IPCA).

Este relatório objetiva documentar o processo de elaboração do Projeto Gestão de Antenas. O Projeto consiste no desenvolvimento de um programa em C, objetivando a manipulação de estruturas de dados dinâmicas, armazenamento em ficheiro, modularização e apresentar uma estruturação e documentação com Doxygen. Pretendo, ainda, desenvolver competências no âmbito da realização de projetos.

A ferramenta Doxygen é amplamente utilizada para gerar documentação a partir de comentários em código fonte, este permite incluir descrições detalhadas de funções, classes e parâmetros diretamente nos arquivos de código, facilitando a manutenção e a legibilidade do Projeto.

Para o desenvolvimento deste trabalho foram aplicados os conhecimentos lecionados nas aulas de EDA, bem como conhecimentos aprendidos nas UC´s lecionadas no primeiro semestre de LESI.

O relatório encontra-se dividido em seis partes, das quais: Introdução, Enquadramento Teórico, Projeto de Gestão de Antenas, Resultados Obtidos, Conclusão e Referências Bibliográficas. Na Introdução, o presente capítulo, é feita a apresentação do contexto, objetivos e estrutura do relatório. O Enquadramento Teórico consagra a revisão da literatura efetuada para a elaboração deste Projeto. O capítulo denominado “Projeto de Gestão de Antenas”, concretiza o corpo do trabalho, onde se aborda a estrutura do Projeto, definição de estruturas de dados, funcionalidades implementadas, descrição dos ficheiros do Projeto e diagramas ilustrativos elaborados ao longo do processo. Na secção dos Resultados Obtidos é feita uma análise da solução deste Projeto e o que este concretiza. Já na Conclusão, temos um resumo dos resultados obtidos e dificuldades sentidas. Por último, as Referências Bibliográficas, onde são apresentadas as bases teóricas utilizadas no desenvolvimento do trabalho.

1. FASE 1- LISTAS LIGADAS

1.1. ENQUADRAMENTO TEÓRICO

A linguagem de programação C foi desenvolvida por Dennis Ritchie nos *Bell Telephone Laboratories* em 1972, com o objetivo de criar uma linguagem eficiente e portátil para o desenvolvimento do sistema operativo Unix. Devido à sua capacidade de manipular diretamente endereços de memória e de realizar operações de baixo nível com grande eficiência, C rapidamente se tornou a linguagem padrão para o desenvolvimento de sistemas, compiladores e aplicações onde o desempenho é essencial (Ritchie, 1993).

É uma linguagem estruturada, que promove o uso de funções modulares e uma sintaxe concisa. Uma das suas grandes vantagens é a portabilidade: um programa em C pode ser facilmente compilado em diferentes arquiteturas com poucas alterações (Kernighan & Ritchie, 1988). A linguagem continua a ser amplamente utilizada na atualidade, sendo a base para muitas outras linguagens modernas, que herdam a filosofia de controlo e desempenho direto da linguagem C.

Os apontadores são uma das características mais poderosas e distintivas da linguagem C. Um apontador é uma variável cujo conteúdo é o endereço de memória de outra variável. Esta funcionalidade confere à linguagem uma flexibilidade única, permitindo que o programador aceda diretamente à memória, manipule estruturas complexas e implemente algoritmos eficientes (Kernighan & Ritchie, 1988).

A declaração de um apontador em C é feita utilizando o operador " * ", indicando que a variável criada irá armazenar um endereço de memória, já o operador "&" é utilizado para obter o endereço de uma variável (King, 2008).

Os apontadores são essenciais para a alocação dinâmica de memória, através das funções da biblioteca <stdlib.h>, como malloc(), calloc(), realloc() e free(). Esta abordagem é vital para a criação de estruturas de dados dinâmicas, como listas ligadas e árvores, cuja dimensão não é conhecida à partida. O programador pode, assim, alocar exatamente a quantidade de memória necessária em tempo de execução (King, 2008).

Além disto, os apontadores são também frequentemente utilizados com estruturas (*structs*), especialmente útil na implementação de estruturas de dados complexas, como listas ligadas, em que cada nó é representado por uma estrutura contendo um ponteiro para o próximo nó (Weiss, 2013).

As listas ligadas são uma estrutura de dados dinâmica onde cada elemento (nó) aponta para o próximo. Em contraste com os arrays, que requerem alocação de memória contínua e de tamanho fixo, as listas ligadas permitem uma gestão dinâmica de memória, o que facilita inserções e remoções em tempo constante em qualquer posição da lista, desde que se tenha o ponteiro adequado (Weiss, 2013).

Em C, as listas ligadas são geralmente implementadas com a utilização de estruturas (*struct*) que definem os dados de cada nó e um ponteiro para o próximo. Este tipo de estrutura é amplamente utilizado na elaboração de projetos, pois permite um diversificado uso, nomeadamente na construção de matrizes, árvores e grafos. Uma das maiores vantagens das listas ligadas é o uso eficiente da memória, já que se aloca apenas o necessário em tempo de execução. Contudo, apresentam algumas desvantagens, como a dificuldade de acesso direto aos elementos (não existe acesso aleatório como nos arrays), e a complexidade adicional no controlo de memória (Tanenbaum, 2007).

No que concerne à manipulação de ficheiros, esta revela-se uma componente essencial da linguagem C, permitindo a interação com o sistema de ficheiros do sistema operativo. A biblioteca padrão stdio.h fornece um conjunto robusto de funções para leitura e escrita de ficheiros, tais como fopen(), fclose(), fscanf(), fprintf(), fread() e fwrite(). Estas funções permitem abrir ficheiros em diferentes modos (leitura, escrita, binário, etc.), ler e escrever dados estruturados e realizar operações eficientes de acesso sequencial e aleatório (Kochan, 2014).

A manipulação eficiente de ficheiros é essencial para persistência de dados, leitura de grandes volumes de informação e comunicação com dispositivos de entrada/saída. O uso correto de ficheiros envolve práticas como verificação de erros após a abertura, controlo do ponteiro de ficheiro e libertação de recursos com fclose() (King, 2008).

Além disso, a manipulação de ficheiros em C é muitas vezes usada em conjunto com *system calls*, em contextos onde é necessário um controlo ainda mais direto dos recursos do sistema. As *system calls* (chamadas ao sistema) constituem a fronteira entre os programas em espaço de utilizador e os serviços oferecidos pelo núcleo do sistema operativo. Através destas chamadas, os programas podem solicitar funcionalidades como leitura e escrita de ficheiros, criação e gestão de processos, comunicação entre processos, alocação de memória, entre outros (Silberschatz, Galvin, & Gagne, 2020).

Em sistemas Unix, estas chamadas são disponibilizadas por meio da biblioteca <unistd.h> e funções como open(), read(), write() e close(). Estas funções não fazem parte da biblioteca padrão do C, mas são suportadas em ambientes POSIX, sendo fundamentais para o desenvolvimento de aplicações que interagem diretamente com o sistema operativo (Love, 2010).

Ao utilizar *system calls*, o processo muda do modo utilizador para o modo kernel, o que implica um custo computacional e obriga a garantir segurança e estabilidade. No entanto, oferecem uma interface poderosa para o desenvolvimento de aplicações robustas e eficientes que requerem acesso direto ao sistema.

1.2. PROJETO DE GESTÃO DE ANTENAS

1.2.1 Objetivos do Projeto

O principal objetivo deste projeto é dar resposta à componente de avaliação da UC EDA. Em suma, o projeto consistiu em desenvolver código fonte em C, com estruturas organizadas e funcionalidades bem definidas, objetivando manipulação de listas ligadas simples e utilização de ficheiros. No contexto do problema (ANEXO I), pretende-se desenvolver uma solução capaz de inserir de uma nova antena na lista ligada, remover uma antena da mesma, deduzir as localizações com efeito nefasto e representá-las, bem como listar na consola de forma tabular as antenas e localizações de efeito nefasto.

1.2.2 Estrutura do Projeto

O projeto foi organizado de forma modular, seguindo as boas práticas de programação em C, separando o código por responsabilidade e facilitando a manutenção e expansão do sistema, sendo ainda possível a reutilização do código. Deste modo, este Projeto foi subdividido da forma considerada mais pertinente tendo em conta o objetivo proposto, os seguintes subcapítulos representam os ficheiros do mesmo.

1.2.2.1 estruturas.h

Ficheiro de cabeçalho onde é definida a estrutura de dados principal. Define a estrutura da antena. Contem a frequência da antena, coordenadas da antena (x e y) e um ponteiro para a próxima antena (formando uma lista ligada simples).

Neste é também definida uma estrutura auxiliar utilizada de modo a gravar os dados em ficheiros num formato binário. Gravar dados em formato binário acarreta várias vantagens, nomeadamente: ocupação de menos espaço no disco comparativamente com ficheiros de texto, rapidez de leitura e escrita, preservação exata dos dados entre outros (Figura 1).

```c
/**
 * @brief Estrutura que representa uma antena com frequência e coordenadas.
 */
typedef struct Antena {
    char frequencia;     /** Frequência da antena */
    int x, y;            /** Coordenadas da antena (x, y) */
    struct Antena *prox; /** Ponteiro para a próxima antena na lista ligada */
} Antena;

/*!
 * @brief Estrutura auxiliar para ficheiro binário.
 */
typedef struct {
    char tipo;  // pode ser frequência da antena ou '#' para interferência
    int x;
    int y;
} RegistoBin;
```

Figura 1- Estruturas utilizadas

1.2.2.2 FuncoesAntenas.h

Ficheiro de cabeçalho com os protótipos das funções utilizadas em todo o projeto. Permite ao main.c e outros módulos usarem as funções de manipulação de antenas e de cálculo de interferências (Figura 2).

```
Antena* CriarAntena(char freq, int x, int y);
Antena* InserirAntenaOrdenada(Antena* inicio, Antena* nova);
Antena* ProcurarAntena(Antena* h, char freq, int x, int y);
Antena* RemoverAntena(Antena* h, char freq, int x, int y);
void ListarAntenas(Antena* h);
void CalcularInterferencias(Antena* lista, int linhas, int colunas);
Antena* CarregarAntenasDeFicheiro(const char* nomeFicheiro, int* linhas, int* colunas);
bool GravarAntenasBinario(const char* nomeFicheiro, Antena* h);
```

Figura 2 - Declaração das funções utilizadas no projeto

1.2.2.3 FuncoesAntenas.c

Implementação da criação, inserção, remoção e cálculo das interferências. Ficheiro onde estão as implementações das funções evidenciadas na figura anterior (Figura 2).

Este ficheiro constitui o núcleo funcional do projeto, reunindo a implementação das principais operações sobre a estrutura de dados utilizada: uma lista ligada de antenas. Este módulo permite criar e inserir antenas de forma ordenada, pesquisar e remover antenas específicas, bem como listar todos os elementos armazenados. Para além disso, inclui uma função essencial para o cálculo de interferências entre antenas com a mesma frequência, desenhando os respetivos vetores simétricos numa matriz bidimensional, representando graficamente as interferências através do caractere '#'. Outra funcionalidade importante é a leitura de antenas a partir de ficheiros de texto, onde cada antena é identificada por um caractere distinto de ponto final '.', sendo posteriormente inserida na lista ligada com as coordenadas correspondentes. Por fim, o módulo oferece a capacidade de gravar os dados resultantes (antenas e interferências) num ficheiro binário, permitindo assim o armazenamento persistente da informação processada. Toda esta organização modular facilita a reutilização do código e contribui para uma estrutura clara, robusta e extensível.

1.2.2.4 main.c

Responsável por orquestrar todas as operações, desde carregar dados até apresentar resultados e guardar em ficheiro. O ficheiro main.c é utilizado para invocar as funções e serve como ponto de entrada do programa, tem como principal objetivo orquestrar a execução sequencial das várias funcionalidades implementadas no projeto. Este ficheiro

ilustra de forma clara a sequência lógica do funcionamento do sistema, reunindo todas as operações fundamentais e servindo como base para testes e demonstrações do comportamento do programa.

1.2.2.5 Makefile

A Makefile (Figura 3), automatiza o processo de compilação e execução. Permite as seguintes tarefas:

- Compila os ficheiros .c em ficheiros .o (código máquina);

- Gera o executável final do programa;

- Permite limpar os ficheiros temporários;

```
all: prog

FuncoesAntenas.o: FuncoesAntenas.c FuncoesAntenas.h estruturas.h
    gcc -c FuncoesAntenas.c

prog: FuncoesAntenas.o main.c
    gcc main.c FuncoesAntenas.o -o prog

run:
    ./prog

clean:
```

Figura 3 - Makefile

1.2.3 As Antenas

No contexto deste trabalho, a utilização de listas ligadas permitiu representar de forma eficiente um conjunto dinâmico de antenas lidas a partir de um ficheiro de texto, cujas quantidades e posições não eram previamente conhecidas. Esta abordagem facilitou a gestão da memória e a implementação de operações como inserir, remover e percorrer a lista, além de tornar o código mais modular e adaptável a alterações. Assim, as listas ligadas mostraram-se uma solução eficaz para manipular um conjunto de dados com tamanho variável.

a) Criar Antena

A função malloc (memory allocation) é utilizada para alocar dinamicamente memória durante a execução de um programa, esta alocação dinâmica permite reservar apenas a memória necessária quando ela for realmente utilizada, oferecendo maior flexibilidade e melhor gestão de recursos (Damas, 2015).

Na função criarAntena, o malloc foi utilizado para alocar memória suficiente para armazenar uma nova estrutura do tipo Antena. Como o número de antenas a serem criadas não é conhecido à partida (depende do conteúdo do ficheiro lido), é essencial que cada antena seja alocada individualmente em tempo de execução. Isto garante que o programa consiga criar tantos nós quantos forem necessários, sem desperdiçar memória ou impor limites fixos. Além disso, esta abordagem é indispensável para trabalhar com listas ligadas, já que cada novo nó precisa de existir independentemente dos outros e ser ligado dinamicamente à lista.

Na figura seguinte (Figura 4), podemos observar a função de criação de um novo nodo, uma nova antena.

```c
Antena* CriarAntena(char freq, int x, int y) {
    Antena* nova = (Antena*)malloc(sizeof(Antena));
    if (nova != NULL) {
        nova->frequencia = freq;
        nova->x = x;
        nova->y = y;
        nova->prox = NULL;
    }
    return nova;
}
```

Figura 4 - Função para Criar Antena

b) Inserir Antena

As antenas são guardadas numa lista ligada de forma ordenada, podendo ser inseridas, no início, no fim ou em qualquer elemento da lista (Figura 5).

```c
Antena* InserirAntenaOrdenada(Antena* inicio, Antena* nova) {
    if (!nova || ProcurarAntena(inicio, nova->frequencia, nova->x, nova->y))
        return inicio;

    if (!inicio || (nova->y < inicio->y || (nova->y == inicio->y && nova->x < inicio->x))) {
        nova->prox = inicio;
        return nova;
    }

    Antena *atual = inicio, *anterior = NULL;
    while (atual && (atual->y < nova->y || (atual->y == nova->y && atual->x < nova->x))) {
        anterior = atual;
        atual = atual->prox;
    }
    anterior->prox = nova;
    nova->prox = atual;
    return inicio;
}
```

Figura 5 - Função para Inserir Antenas

c) Remover Antena

```c
Antena* RemoverAntena(Antena* h, char freq, int x, int y) {
    Antena *aux = h, *anterior = NULL;
    while (aux && (aux->frequencia != freq || aux->x != x || aux->y != y)) {
        anterior = aux;
        aux = aux->prox;
    }
    if (!aux) return h;

    if (!anterior)
        h = aux->prox;
    else
        anterior->prox = aux->prox;

    free(aux);
    return h;
}
```

Figura 6 - Remover Antena

Percorre a lista ligada à procura de uma antena com frequência e coordenadas específicas. Se for a primeira (cabeça), atualiza o ponteiro da cabeça. Caso contrário, liga o nó anterior ao seguinte. Por fim, liberta a memória da antena removida (Figura 6).

d) Listar Antenas

Percorre a lista ligada imprimindo a frequência e coordenadas de cada antena. Útil para verificar se a leitura do ficheiro e a inserção na lista ocorreram corretamente (Figura 7).

```c
void ListarAntenas(Antena* h) {
    while (h) {
        printf("Frequência: %c | Coordenadas: (%d, %d)\n", h->frequencia, h->x, h->y);
        h = h->prox;
    }
}
```

Figura 7 - Listar Antenas

e) Procurar Antena

Função simples que percorre a lista ligada em busca de antenas que preencham os critérios definidos nos seus parâmetros (Figura 8).

```c
Antena* ProcurarAntena(Antena* h, char freq, int x, int y) {
    while (h) {
        if (h->frequencia == freq && h->x == x && h->y == y)
            return h;
        h = h->prox;
    }
    return NULL;
}
```

Figura 8 - Procurar Antena

1.2.4 As Interferências

A parte mais desafiadora foi a lógica para detetar interferência entre antenas com a mesma frequência. A ideia baseia-se na criação de um vetor entre duas antenas iguais, e posterior aplicação do mesmo sobre as coordenadas destas antenas.

1.2.4.1 Cálculo do número de interferências

Objetivando perceber melhor a dinâmica do exercício proposto no projeto, foram elaborados diversos esquemas. Na imagem seguinte (Figura 9), é possível observar o raciocínio que despoletou a descoberta do número de interferência, tendo em conta o número de antenas.

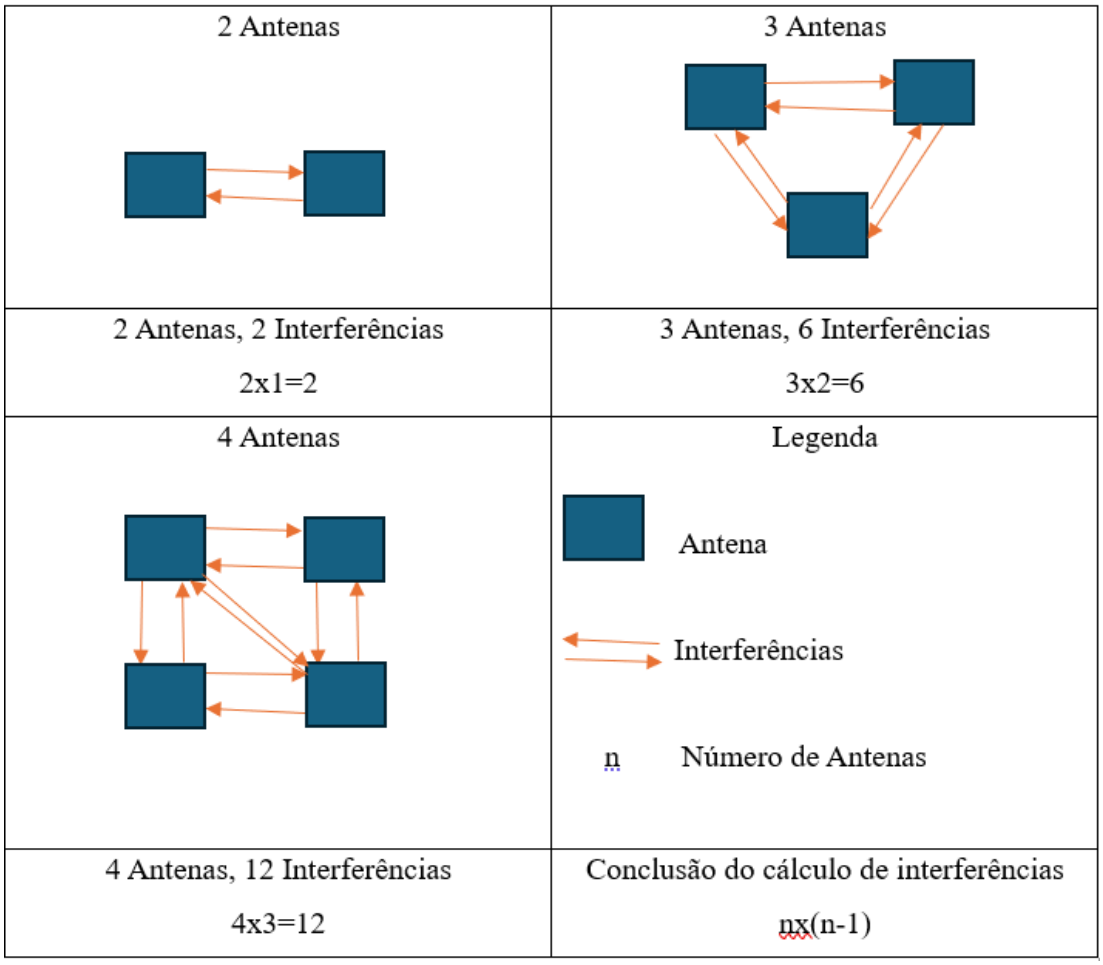


Figura 9 - Cálculo do número de interferências

1.2.4.2 Posicionamento das interferências

Após a interpretação cuidadosa do enunciado e ter obtido compreensão acerca do número de interferências e como este variava consoante a introdução de novas antenas foi

formulada uma solução que fosse capaz de determinar a posição sobre a qual recairiam os efeitos nefastos referentes às antenas.

Assumiremos que a primeira antena encontrada aquando da leitura da matriz de cima para baixo se irá denominar por A1, e a segunda antena dessa mesma frequência por A2. Deste modo obtive a seguinte conclusão, calcula-se o vetor $v = A2 - A1$ e posteriormente as interferências ocorrem em:

• P1 = A1 $-v$

• P2 = A2 $+v$

	1	2	3	4	5	6	7	8	9	10	11
1		#									
2											
3			A1	#	#						
4											
5				A2							
6											
7		#	A3	A4	#						
8											
9				#							
10											
11			#		#						

Figura 10 - Exemplo cálculo Interferências

Como podemos constatar várias vezes na figura anterior (Figura 10), aplicando as operações vetoriais mencionadas anteriormente, temos por exemplo:

A2-A1=(5,4)-(3,3)=(2,1);

Resultando no vetor v=(2,1).

Quando a A1, a primeira antena a surgir aquando da leitura da matriz de cima para baixo, subtraímos o vetor *v,* obtemos:

A1 – v = (3,3) – (2,1)=(1,2), como é possível evidenciar na figura acima;

De modo a obter a outra interferência gerada entre A1 e A2, iremos somar às coordenadas de A2 o vetor *v:*

A2 + v= (5,4) +(2,1)= (7,5), conforme evidenciado na figura.

1.2.4.3 Concretização do algoritmo para detetar interferências

Deste modo, e após interpretação de forma exaustiva foi idealizado este algoritmo com o intuito de responder ao problema solicitado anteriormente. Primeiramente as antenas seriam carregadas de um ficheiro em formato txt, sendo inseridas na lista ligada de forma ordenada. Posteriormente seria aplicado este algoritmo de modo a obter a localização das interferências, sendo posteriormente gravadas as posições das antenas, assim como a posição dos efeitos nefastos. É também exibida na consola como a matriz ficaria com a posição das interferências. Esta solução pode ser observada na figura seguinte (Figura 11):

```c
void CalcularInterferencias(Antena* lista, int linhas, int colunas) {
    char matriz[linhas + 1][colunas + 1];
    // Inicializar a matriz com '.'
    for (int y = 1; y <= linhas; y++) {
        for (int x = 1; x <= colunas; x++) {
            matriz[y][x] = '.';
        }
    }
    // Colocar as antenas na matriz
    for (Antena* a = lista; a != NULL; a = a->prox) {
        if (a->x >= 1 && a->x <= colunas && a->y >= 1 && a->y <= linhas)
            matriz[a->y][a->x] = a->frequencia;
    }
    // Calcular interferências com base em pares de antenas
    for (Antena* a1 = lista; a1 != NULL; a1 = a1->prox) {
        for (Antena* a2 = a1->prox; a2 != NULL; a2 = a2->prox) {
            if (a1->frequencia == a2->frequencia) {
                int dx = a2->x - a1->x;
                int dy = a2->y - a1->y;
                // A1 - V
                int x1 = a1->x - dx;
                int y1 = a1->y - dy;
                if (x1 >= 1 && x1 <= colunas && y1 >= 1 && y1 <= linhas && matriz[y1][x1] == '.')
                    matriz[y1][x1] = '#';
                // A2 + V
                int x2 = a2->x + dx;
                int y2 = a2->y + dy;
                if (x2 >= 1 && x2 <= colunas && y2 >= 1 && y2 <= linhas && matriz[y2][x2] == '.')
                    matriz[y2][x2] = '#';
            }
        }
    }
    // Mostrar matriz;
printf("\nMatriz com interferências (formato original do ficheiro):\n");
for (int y = 1; y <= linhas; y++) {
    for (int x = 1; x <= colunas; x++) {
        printf("%c", matriz[y][x]);
    }
    printf("\n");
}
}
```

Figura 11 - Calcular interferências

1.2.5 Manipulação de ficheiros

Carregar dados de ficheiros .txt e gravar ficheiros em formato binário são práticas fundamentais em programação, com finalidades diferentes, mas complementares.

Os Ficheiros .txt são fáceis de abrir e entender com qualquer editor de texto, sendo portante ideais para partilhar dados ou importar/exportar entre sistemas. O facto de ser possível visualizar o conteúdo dos mesmos também ajuda no processo de *Debug*, devido ao facto de ajudar a identificar erros.

Após o cálculo das interferências optamos então por gravar os dados obtidos em formato binário, usufruindo das suas múltiplas vantagens, nomeadamente um acesso mais rápido, devido ao facto de não ocorrerem conversões entre texto e tipos de dados, uma ocupação menor de espaço, e ainda adiciona uma camada de segurança devido ao facto destes dados serem menos legíveis.

Seguem as funções elaboradas de modo a colmatar estas necessidades enunciadas (Figura 12 e Figura 13):

```c
Antena* CarregarAntenasDeFicheiro(const char* nomeFicheiro, int* linhas, int* colunas) {
    FILE* f = fopen(nomeFicheiro, "r");
    if (!f) {
        perror("Erro ao abrir o ficheiro");
        return NULL;
    }
    printf("Ficheiro aberto com sucesso. A ler linhas...\n");
    char linha[512];
    int numLinhas = 0;
    int maxColunas = 0;
    Antena* lista = NULL;
    while (fgets(linha, sizeof(linha), f)) {
        int comprimento = 0;
        for (int i = 0; linha[i] != '\0' && linha[i] != '\n'; i++) {
            char ch = linha[i];
            if (ch != '.') {
                printf("Antena encontrada: %c em (%d, %d)\n", ch, numLinhas + 1, i + 1);
                Antena* nova = CriarAntena(ch, i + 1, numLinhas + 1);
                lista = InserirAntenaOrdenada(lista, nova);
            }
            comprimento++;
        }
        if (comprimento > maxColunas) maxColunas = comprimento;
        numLinhas++;
    }
    fclose(f);
    *linhas = numLinhas;
    *colunas = maxColunas;
    if (!lista) {
        printf("Nenhuma antena foi carregada do ficheiro!\n");
    }
    return lista;
}
```

Figura 12 - CarregarAntenaDe Ficheiro

```
bool GravarAntenasBinario(const char* nomeFicheiro, Antena* lista) {
    if (!lista) return false;
    FILE* fp = fopen(nomeFicheiro, "wb");
    if (!fp) return false;
    // Determinar tamanho da matriz
    int maxX = 0, maxY = 0;
    for (Antena* a = lista; a != NULL; a = a->prox) {
        if (a->x > maxX) maxX = a->x;
        if (a->y > maxY) maxY = a->y;
    }
    // Criar e inicializar matriz
    char matriz[maxY + 1][maxX + 1];
    for (int y = 1; y <= maxY; y++)
        for (int x = 1; x <= maxX; x++)
            matriz[y][x] = '.';
    // Colocar antenas
    for (Antena* a = lista; a != NULL; a = a->prox) {
        matriz[a->y][a->x] = a->frequencia;
    }
    // Calcular interferências
    for (Antena* a1 = lista; a1 != NULL; a1 = a1->prox) {
        for (Antena* a2 = a1->prox; a2 != NULL; a2 = a2->prox) {
            if (a1->frequencia == a2->frequencia) {
                int dx = a2->x - a1->x;
                int dy = a2->y - a1->y;
                int x1 = a1->x - dx;
                int y1 = a1->y - dy;
                int x2 = a2->x + dx;
                int y2 = a2->y + dy;
                if (x1 >= 1 && x1 <= maxX && y1 >= 1 && y1 <= maxY && matriz[y1][x1] == '.')
                    matriz[y1][x1] = '#';

                if (x2 >= 1 && x2 <= maxX && y2 >= 1 && y2 <= maxY && matriz[y2][x2] == '.')
                    matriz[y2][x2] = '#';
            }
        }
    }
    // Gravar tudo o que não for '.' no binário
    RegistoBin reg;
    for (int y = 1; y <= maxY; y++) {
        for (int x = 1; x <= maxX; x++) {
            if (matriz[y][x] != '.') {
                reg.tipo = matriz[y][x];
                reg.x = x;
                reg.y = y;
                fwrite(&reg, sizeof(RegistoBin), 1, fp);
            }
        }
    }
    fclose(fp);
    return true;
}
```

Figura 13 - GravarAntenasBinario

1.3. RESULTADOS OBTIDOS

Durante a fase de testes do programa, foram realizadas várias experiências com ficheiros de entrada distintos, contendo antenas com diferentes localizações e frequências. O objetivo principal foi validar o correto funcionamento do sistema de leitura dos ficheiros, a inserção ordenada das antenas na lista ligada, o cálculo das interferências entre antenas com a mesma frequência e a criação da matriz de saída que representa graficamente essas interferências. Além disto, pretende-se perceber se o programa é capaz de receber matrizes de tamanhos variáveis.

Num primeiro teste (Figura 14), foi utilizado um ficheiro com várias antenas da frequência '0' distribuídas em posições diferentes da matriz, de coordenadas: (2,9), (3,6), (4,8) e (5,5). Além disso, foram incluídas duas antenas da frequência 'A' em (6,7) e (9,9). O resultado gerado pelo programa foi uma matriz de 9 linhas por 12 colunas, onde se observaram diversos pontos de interferência assinalados com o caractere '#', localizadas nas zonas esperadas em relação aos pares de antenas com a mesma frequência. Este teste confirmou que o programa calcula corretamente as posições de interferência com base no vetor que liga duas antenas idênticas e que estas são representadas visualmente na matriz final.

```
Antena encontrada: 0 em (2, 9)
Antena encontrada: 0 em (3, 6)
Antena encontrada: 0 em (4, 8)
Antena encontrada: 0 em (5, 5)
Antena encontrada: A em (6, 7)
Antena encontrada: A em (9, 9)
Antenas carregadas da matriz (9 linhas x 12 colunas):
Frequência: 0 | Coordenadas: (2, 9)
Frequência: 0 | Coordenadas: (3, 6)
Frequência: 0 | Coordenadas: (4, 8)
Frequência: 0 | Coordenadas: (5, 5)
Frequência: A | Coordenadas: (6, 7)
Frequência: A | Coordenadas: (9, 9)

Matriz com interferências (formato original do ficheiro):
......#....#
...#....0...
....#0....#.
..#....0....
....0....#..
.#....A.....
...#........
#...........
........A...
```

Figura 14 - Teste 1

Num segundo teste (Figura 15), o número de antenas foi reduzido para apenas três: duas da frequência 'O' em (2,9) e (3,6), e uma da frequência 'A' em (9,9). A matriz resultante mostrou apenas duas interferências entre as antenas da frequência 'O', não havendo qualquer interação com a antena da frequência 'A'. Este comportamento está de acordo

com o que se esperava, uma vez que o algoritmo apenas considera pares de antenas com a mesma frequência para o cálculo das interferências.

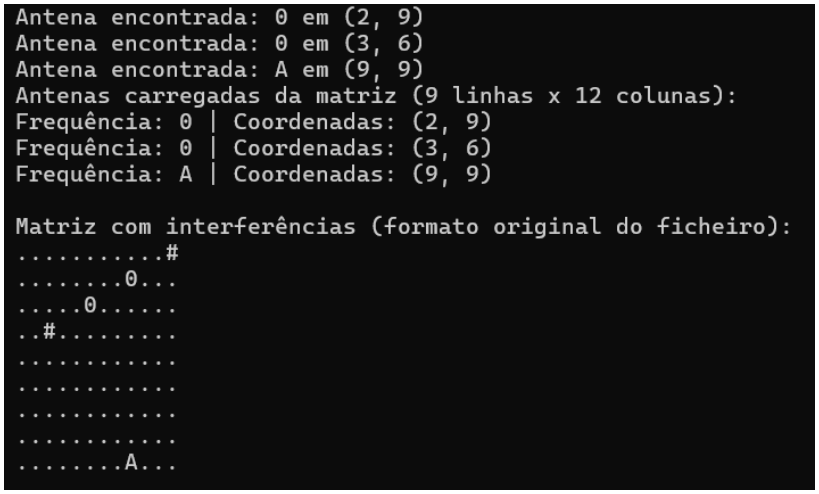


Figura 15 - Teste 2

Num terceiro teste (Figura 16), foi testada uma matriz de dimensão maior, com dimensões 24 colunas e 8 linhas em que as antenas posicionadas são todas com a mesma frequência 'A'. Estavam localizadas nas posições (3,14), (4,7) e (5,16). A matriz resultante evidenciou que, mesmo com uma maior dispersão espacial, o algoritmo foi capaz de identificar os vetores de interferência e posicioná-los corretamente. Este teste foi importante para validar a robustez do cálculo em matrizes de maior dimensão, bem como perceber que o programa é capaz de receber matrizes de tamanho variável.

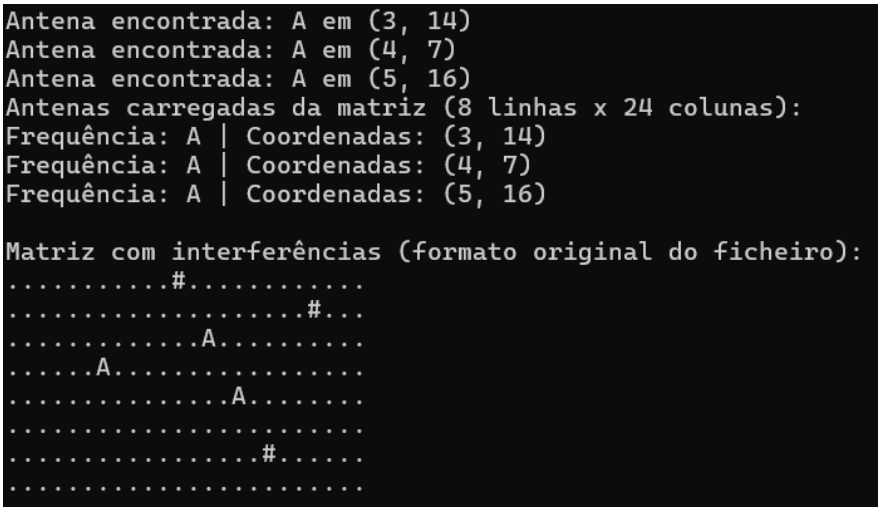


Figura 16 - Teste 3

Por fim, realizou-se um último teste (Figura 17), onde foram colocadas duas antenas, cada uma com uma frequência distinta ('A' em (3,7) e 'O' em (7,3)). Como previsto, não foram

detetadas interferências na matriz, demonstrando que o programa apenas considera pares com frequências iguais, evitando falsos positivos. Além disto, confirma, mais uma vez, que o programa recebe matrizes de vários tamanhos.

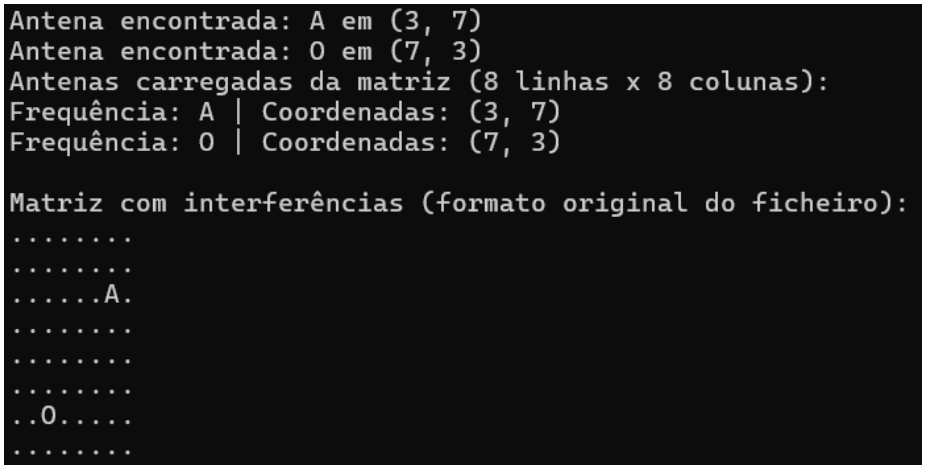


Figura 17 - Teste 4

Em todos os testes realizados, o programa cumpriu os requisitos definidos, produzindo matrizes corretas com antenas e interferências claramente representadas. As antenas foram lidas com sucesso a partir dos ficheiros de texto e inseridas de forma ordenada na lista ligada, permitindo a correta reconstrução da matriz para posterior visualização. Foi possível concluir, ainda, que o programa é capaz de receber matrizes de tamanhos variáveis, o que o torna versátil.

2. FASE 2 – GRAFOS

As estruturas de dados de grafos são fundamentais em ciência de computação, pois permitem representar e manipular relações complexas entre objetos. Um grafo é composto por vértices (ou nós) e arestas (ou ligações) que modelam conexões entre pares de entidades. Esta estrutura é amplamente utilizada para resolver problemas como rotas em mapas, redes de comunicação, dependências entre tarefas, análise de redes sociais, entre outros (Cormen et al., 2009). Em C, os grafos são frequentemente implementados com base em arrays, listas ligadas ou estruturas mais especializadas, como listas ou matrizes de adjacência, dependendo da eficiência desejada para operações como inserção, remoção ou procura (Sedgewick & Wayne, 2011).

Na linguagem C, que se caracteriza por uma gestão manual de memória e ausência de estruturas de dados nativas de alto nível, a implementação de grafos exige um controlo cuidadoso dos ponteiros e da alocação dinâmica. Por esse motivo, as listas ligadas de adjacência são amplamente utilizadas para representar grafos, pois oferecem uma boa relação entre consumo de memória e desempenho (Knuth, 1997). Esta representação permite executar algoritmos clássicos como DFS (Depth-First Search) e BFS (Breadth-First Search) de forma eficiente, mesmo em grandes grafos.

A ausência de abstrações como listas genéricas ou mapas associativos em C, ao contrário do que existe em linguagens como Python ou Java, força o programador a implementar manualmente cada estrutura — o que, embora mais trabalhoso, favorece uma compreensão aprofundada dos conceitos algorítmicos e da gestão de recursos (Kernighan & Ritchie, 1988). Tal domínio é particularmente importante em contextos de baixo nível ou sistemas com restrições de hardware, como dispositivos embebidos ou aplicações em tempo real.

Em suma, a estrutura de grafos é essencial na linguagem C não apenas pelo seu valor algorítmico, mas também pela forma como obriga o programador a compreender e otimizar a representação e manipulação de relações complexas entre dados (Cormen et al., 2009; Knuth, 1997).

O uso de ficheiros de texto (.txt) e ficheiros binários (.bin) representa duas abordagens distintas, mas complementares para o armazenamento e intercâmbio de dados em programação. Os ficheiros de texto são amplamente utilizados pela sua simplicidade e legibilidade. Permitem armazenar informação em formato ASCII ou UTF-8, sendo

facilmente editáveis por humanos e compatíveis com editores genéricos. Este tipo de ficheiros é ideal para configurações, logs, tabelas simples e prototipagem de dados. Já os ficheiros binários guardam os dados em formato bruto, tal como representados na memória, o que permite maior eficiência em termos de espaço e velocidade de leitura/escrita — particularmente útil para estruturas complexas, como listas ligadas, árvores ou grafos (Kernighan & Ritchie, 1988).

Na linguagem C, os ficheiros de texto são normalmente manipulados com funções como fscanf, fprintf ou fgets, enquanto ficheiros binários são tratados com fread e fwrite, que trabalham diretamente sobre blocos de memória. A escolha entre um formato e outro depende dos requisitos da aplicação: ficheiros .txt oferecem portabilidade e transparência; ficheiros .bin, desempenho e integridade da estrutura interna (Silberschatz et al., 2018). Além disso, o uso de ficheiros binários é fundamental em aplicações como jogos, simulações científicas, bases de dados ou comunicações em redes, onde a performance e o tamanho dos dados são críticos (Tanenbaum & Bos, 2015).

Trabalhar corretamente com ambos os formatos desenvolve no programador uma compreensão mais profunda da representação de dados em baixo nível, promove a modularidade e permite a construção de sistemas mais robustos, persistentes e interoperáveis. Em ambientes industriais e académicos, esta é uma competência indispensável para lidar com armazenamento eficiente, serialização de estruturas e integração com outras ferramentas.

Os algoritmos de pesquisa em grafos, como a busca em profundidade - Depth-First Search (DFS) e a busca em largura - Breadth-First Search (BFS), são fundamentais para a análise e resolução de problemas em estruturas relacionais. Estes algoritmos permitem explorar conexões entre entidades de forma sistemática, sendo aplicados em redes de computadores, caminhos mínimos em mapas, análise de dependências, deteção de ciclos, inteligência artificial e muito mais (Cormen et al., 2009). Na linguagem C, estas pesquisas são frequentemente implementadas utilizando recursividade, especialmente no caso do DFS, devido à sua natureza exploratória e à elegância da abordagem baseada em chamadas sucessivas de função. A recursividade permite representar o avanço profundo numa ramificação do grafo sem o uso explícito de estruturas auxiliares como pilhas, facilitando a legibilidade e a correspondência direta com o modelo teórico (Knuth, 1997).

A recursividade, além de ser um conceito central em ciência de computação, é extremamente eficaz na resolução de problemas que têm uma estrutura auto-similar ou divisível em subproblemas semelhantes. Em grafos, essa abordagem permite que o código espelhe diretamente a lógica matemática dos percursos e facilite a implementação de algoritmos como DFS, *backtracking* ou algoritmos de procura de caminhos. No entanto, a sua utilização exige cuidados em C, onde não há controlo automático sobre profundidade de chamadas e gestão de *stack,* o que pode levar a estouros de pilha em grafos muito grandes. Ainda assim, quando bem utilizada, a recursividade é uma ferramenta poderosa que promove soluções elegantes e modulares (Sedgewick & Wayne, 2011).

Assim, os algoritmos de pesquisa em grafos e a recursividade estão intrinsecamente ligados, sendo ambos pilares fundamentais para a construção de software eficiente e capaz de explorar relações complexas entre dados. A sua compreensão e domínio são essenciais na formação de qualquer programador com interesse em estruturas de dados e algoritmia avançada.

A utilização de bibliotecas no desenvolvimento de aplicações com grafos é fundamental para otimizar o trabalho do programador, tanto em termos de desempenho como de produtividade. Em linguagens como C, onde a gestão de memória e a construção de estruturas de dados são tarefas manuais e suscetíveis a erros, as bibliotecas oferecem um nível de abstração que reduz significativamente a complexidade do código. Em vez de implementar manualmente listas de adjacências, matrizes de ligação ou operações de inserção e remoção de vértices e arestas, o programador pode recorrer a funções já prontas e testadas, que executam estas operações de forma eficiente e segura.

As bibliotecas estáticas são coleções de funções, estruturas e dados pré-compilados que podem ser ligadas ao programa no momento da compilação, sendo incorporadas diretamente no executável final. A sua utilização permite uma melhor organização modular do código, facilita a reutilização de funcionalidades comuns e promove a manutenibilidade do software. Em projetos de grande escala, a divisão do código em múltiplos módulos e a compilação de partes estáveis em bibliotecas estáticas reduz significativamente o tempo de compilação incremental, além de contribuir para uma estrutura mais limpa e reutilizável (King, 2008).

Ao contrário das bibliotecas dinâmicas, as bibliotecas estáticas não requerem a presença de ficheiros externos no tempo de execução, o que torna a distribuição do programa mais simples e auto-contida — uma vantagem importante em ambientes embebidos, sistemas de tempo real e software de baixo nível, como drivers ou ferramentas de sistemas operativos (Franke, 2014). Além disso, em C, o uso de bibliotecas estáticas (ficheiros .a no Unix/Linux) permite que o programador encapsule funcionalidades como algoritmos, estruturas de dados ou utilitários sem necessidade de recompilar tudo a cada alteração em pequenos módulos.

A prática de separar a lógica do programa principal em bibliotecas estáticas também favorece a testabilidade do código, pois permite compilar e testar unidades específicas de forma isolada (unit testing). Em contexto académico, este tipo de modularização é fortemente recomendado para promover boas práticas de engenharia de software e para facilitar a avaliação de partes distintas do projeto (Kernighan & Pike, 1999).

Em resumo, as bibliotecas para grafos oferecem um conjunto robusto de funcionalidades que simplificam a implementação, aumentam a eficiência, melhoram a manutenção do código e garantem resultados mais fiáveis. Ao recorrer a estas ferramentas, o programador pode focar-se na lógica da aplicação e nos objetivos do projeto, deixando as tarefas estruturais e algorítmicas para componentes já consolidadas e testadas.

2.1. PROJETO DE GESTÃO DE ANTENAS

2.1.1 Objetivos do Projeto

O principal objetivo deste Projeto é dar resposta à componente de avaliação da UC EDA. O Projeto consistiu em desenvolver um programa em C, com estruturas organizadas e funcionalidades bem definidas, objetivando aplicar conceitos avançados de teoria dos grafos para resolver o mesmo problema computacional. No contexto do problema (ANEXO II), pretende-se que cada antena seja modelada como um vértice, sendo interligada a outras com a mesma frequência através de arestas. O projeto deverá permitir o carregamento automático dos dados de antenas a partir de um ficheiro de texto, a construção do grafo correspondente e a implementação de operações de exploração como a procura em profundidade (DFS), em largura (BFS), identificação de todos os caminhos possíveis entre duas antenas e deteção de interseções entre antenas com frequências distintas.

2.1.2 Estrutura do Projeto

O projeto foi organizado de forma modular, seguindo as boas práticas de programação em C, separando o código por responsabilidade e facilitando a manutenção e expansão do sistema, sendo ainda possível a reutilização do código. Deste modo, este Projeto foi subdividido da forma considerada mais pertinente tendo em conta o objetivo proposto, os seguintes subcapítulos representam os ficheiros do mesmo.

2.1.2.1 grafo_structs.h

O ficheiro grafo_structs.h define as estruturas de dados fundamentais que suportam todo o funcionamento do projeto. Nele encontramos a definição dos vértices (Vertice), que representam antenas identificadas por coordenadas (linha e coluna) e uma frequência. Cada vértice aponta para uma lista ligada de adjacências (Adjacencia), representando as ligações diretas a outras antenas. Adicionalmente, a estrutura Interferencia permite armazenar posições onde ocorre interferência de sinal, sendo também representada por uma lista ligada.

2.1.2.2 grafo_funcoes.h/.c

A manipulação das estruturas mencionadas anteriormente é realizada nos ficheiros grafo_funcoes.h e grafo_funcoes.c, que contêm as funções de criação, inserção e gestão do grafo. Estas incluem a criação de vértices e adjacências, inserção em listas ligadas, listagem de todas as antenas presentes no grafo, cálculo de interferências entre antenas com a mesma frequência, e libertação de memória ocupada pelas estruturas. Um dos pontos-chave deste módulo é a representação do grafo como uma lista ligada de vértices, o que permite uma abordagem flexível e adaptável a grafos esparsos.

Neste ficheiro é também possível observar um dos componentes mais interessantes do projeto, que tratam da deteção geométrica de interseções entre adjacências. A função intersecoesParametricas() baseia-se em equações paramétricas de segmentos de reta e resolve um sistema linear para determinar se dois segmentos se cruzam e em que ponto. Esta lógica é integrada na função detetarIntersecoes(), que percorre todas as combinações possíveis de adjacências no grafo, ignorando pares redundantes, e imprime os cruzamentos detetados entre segmentos de frequências diferentes.

2.1.2.3 – ficheiros.h/.c

Complementando esta estrutura, os ficheiros: ficheiros.h e ficheiros.c tratam da persistência dos dados, isto é, da leitura e escrita em ficheiros. A função carregarMatriz() interpreta um ficheiro .txt contendo a matriz das antenas e converte cada célula não vazia num vértice. As funções gravarGrafoBincomInterferencias() e lerGrafoBinario() permitem gravar e restaurar o grafo e a lista de interferências a partir de ficheiros binários, garantindo assim a persistência e portabilidade da estrutura de dados, mesmo entre diferentes sessões de execução.

2.1.2.4 – procura.h/.c

Os algoritmos de exploração do grafo estão implementados nos ficheiros procura.h e procura.c. Estes incluem as funções clássicas de busca em grafos, como DFS (Depth-First Search) e BFS (Breadth-First Search), que percorrem o grafo a partir de um vértice inicial. A DFS é implementada de forma recursiva e visita os vértices de forma profunda antes de retroceder, enquanto a BFS utiliza uma fila para percorrer os vértices em largura.

2.1.2.5 main.c

Responsável por orquestrar todas as operações, desde carregar dados até apresentar resultados e guardar em ficheiro. O ficheiro main.c é utilizado para invocar as funções e serve como ponto de entrada do programa, tem como principal objetivo orquestrar a execução sequencial das várias funcionalidades implementadas no projeto. Este ficheiro ilustra de forma clara a sequência lógica do funcionamento do sistema, reunindo todas as operações fundamentais e servindo como base para testes e demonstrações do comportamento do programa.

2.2 As Interseções

A parte mais desafiadora desta segunda parte do projeto foi a lógica por trás do cálculo das interseções entre arestas. Num grafo de antenas, as ligações entre antenas (arestas) podem ser representadas geometricamente como segmentos de reta. Verificar se duas dessas ligações se cruzam é essencial, por exemplo, para detetar conflitos físicos entre cabos ou frequências em planos de cobertura. O cálculo de interseções entre segmentos envolve determinar se existe um ponto comum entre dois segmentos distintos, e caso exista, onde ele ocorre. Deste modo foi implementada esta lógica em duas funções de modo a responder ao problema do cálculo dos pontos de interseção entre as diversas

arestas, deste modo são apresentados em seguida os raciocínios matemáticos e algébricos que conduziram à elaboração destas funções.

Existem três formas de representar retas e segmentos de reta, sendo eles, a equação reduzida da reta (y=mx + b), a equação vetorial ((x,y)=(x0,y0)+t(a,b), k ∈ R, e ainda a equação paramétrica da reta, cuja expressão é algo do género:

$$r(t) = \begin{cases} f(t) = x_0 + at \\ g(t) = y_0 + bt \end{cases}$$

Figura 18 - Expressão da equação paramétrica da reta

Estas equações representam qualquer segmento de reta no plano, funcionando naturalmente com intervalos t∈[0,1], permitem limitar o domínio, ou seja, distinguir segmentos de retas infinitas e são adaptadas para a resolução de interseções como sistema de equações. O valor de t diz onde está o ponto ao longo do segmento.

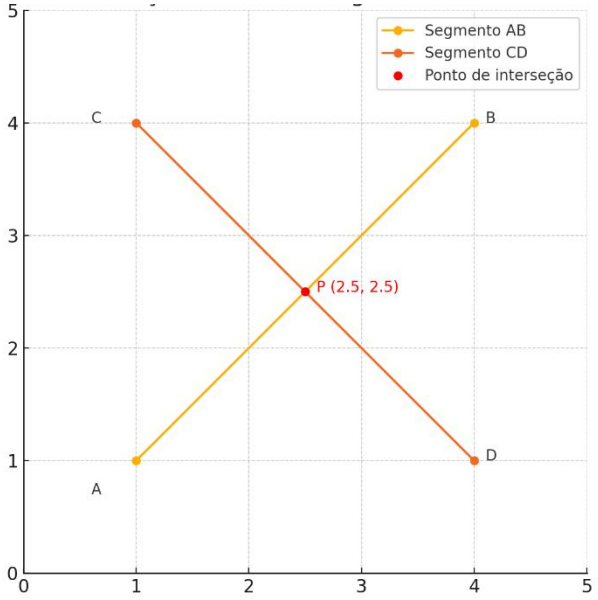


Figura 19 - Interseção de dois segmentos de reta

No gráfico representado t=0 representa o início do segmento (ponto A), sendo que t=1 representa o fim do segmento (ponto B), deste modo qualquer ponto entre A e B estará representado no intervalo [0,1] e qualquer valor fora deste domínio representa um ponto fora do segmento, numa extensão da reta.

Cada segmento de reta pode ser representado por uma equação paramétrica da forma:

- Para o segmento AB:

$$P_1(t) = A + t(B - A), \quad \text{com } 0 \leq t \leq 1$$

Figura 20 - Equação paramétrica do segmento AB

- Para o segmento CD:

$$P_2(u) = C + u(D - C), \quad \text{com } 0 \leq u \leq 1$$

Figura 21 - Equação paramétrica do segmento CD

O ponto de interseção ocorre quando fazemos a igualdade entre as expressões:

$$A + t(B - A) = C + u(D - C)$$

Figura 22 - Igualdade entre as duas expressões

Traduzindo esta expressão para as coordenadas dos pontos temos:

$$x_1 + t(x_2 - x_1) = x_3 + u(x_4 - x_3)$$

$$y_1 + t(y_2 - y_1) = y_3 + u(y_4 - y_3)$$

Figura 23 - Aplicação das coordenadas dos pontos

Este sistema de duas equações com duas incógnitas (t e u) pode ser resolvido com álgebra linear.

Obtendo assim duas equações lineares com duas incógnitas, t e u. Organizando as equações como um sistema, temos:

$$\begin{cases} t(x_2 - x_1) - u(x_4 - x_3) = x_3 - x_1 \\ t(y_2 - y_1) - u(y_4 - y_3) = y_3 - y_1 \end{cases}$$

Figura 24 - Organização das equações lineares

Podendo ser reescrito da forma:

$$\begin{cases} at + bu = c \\ dt + eu = f \end{cases}$$

Figura 25 - Simplificação das equações lineares

Onde:

- a=x2−x1;
- b=−(x4−x3);
- c=x3−x1;
- d=y2−y1;
- e=−(y4−y3);
- f=y3−y1;

De modo a dar resposta ao problema colocado foram utilizados conceitos lecionadas nas aulas de Álgebra Linear, tais como a Regra de Cramer (ou regra dos determinantes), de modo a resolver o sistema de equações lineares que surge da interseção entre dois segmentos de reta apresentados pelas equações paramétricas.

Deste modo, colocando na forma matricial temos:

$$\begin{bmatrix} a & b \\ d & e \end{bmatrix} \cdot \begin{bmatrix} t \\ u \end{bmatrix} = \begin{bmatrix} c \\ f \end{bmatrix}$$

Figura 26 - Forma matricial

A Regra de Cramer é uma técnica de álgebra linear usada para resolver sistemas lineares de equações — mais precisamente, sistemas com o mesmo número de equações e incógnitas (normalmente pequenos, como 2×2 ou 3×3). O seu objetivo é: Encontrar os valores das variáveis (como t e u) num sistema linear de forma direta e exata, usando apenas determinantes.

Neste projeto esta regra auxilia-nos a resolver uma matriz 2x2 que representa a interseção de dois segmentos de reta, a obter os valores dos parâmetros t e u, que determinam se os segmentos se cruzam, verificar se t∈[0,1] e u∈[0, 1] de modo a compreender se a interseção ocorre dentro dos segmentos, e não fora. Em caso positivo auxiliar no cálculo do ponto de interseção e na deteção de colisões/interferências visuais entre as ligações (arestas).

Posteriormente são efetuados e demonstrados os cálculos dos determinantes:

O determinante principal (D), sendo zero, significa que o sistema não tem solução única, ou seja, as retas são paralelas ou coincidentes, este é obtido da seguinte forma:

$$D = \begin{vmatrix} a & b \\ d & e \end{vmatrix} = a \cdot e - b \cdot d$$

Figura 27 - Cálculo do determinante principal

Posteriormente são calculados o determinante de t (Det(t) ou Dt), e o determinante de u (Det(u) ou Du) da seguinte forma:

$$D_t = c \cdot e - b \cdot f$$

Figura 28 - Cálculo do determinante de t

$$D_u = a \cdot f - c \cdot d$$

Figura 29 - Cálculo do determinante de u

De modo a ser mais fácil utilizar a regra de Cramer e posteriormente calcular o determinante de t e u, a última coluna é substituída pela primeira e efetuado o cálculo do determinante como exemplificado anteriormente e posteriormente pela segunda e efetuado o mesmo cálculo

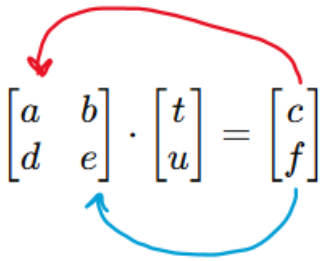


Figura 30 - Demonstração do cáculo de Dt e Du

Posteriormente, de modo a obter o valor das variáveis são efetuados os seguintes cálculos:

$$t = \frac{D_t}{D}$$

Figura 32 - Cálculo da variável t

$$u = \frac{D_u}{D}$$

Figura 31 - Cálculo da variável u

Depois de calcular t e u, se ambos se encontrarem no intervalo [0,1] garantimos que a interseção acontece dentro dos segmentos.

Estes parâmetros t e u controlam onde nos encontramos ao longo do segmento de reta, podendo assumir qualquer valor real, contudo apenas é proveitoso para este caso quando se encontram entre zero e um. Logo se o valor de ambos os parâmetros se encontrarem

neste intervalo, após ter sido feito a igualdade entre as equações paramétricas, isto significa que existe interseção ao longo dos dois segmentos de reta.

De modo a tornar mais simples a interpretação destes raciocínios e destes conceitos é efetuada uma demonstração de um caso prático. Se tivermos:

- A = (1, 1), B = (4, 4)
- C = (1, 4), D = (4, 1)

Então:

- a=3; b=−3; c=0
- d=3; e=3; f=3

Aplicamos:

- D=3·3−(−3)·3=9+9=18;
- Dt=0·3−(−3)·3=0+9=9;
- Du=3·3−0·3=9;

Então:

- t=9/18=0.5t
- u=9/18=0.5

Deste modo conseguimos comprovar que existe interseção entre estas arestas pois ambos se encontram entre 0 e 1.

Posteriormente é efetuado o cálculo do ponto de interseção substituindo os valores de t e u na expressão.

2.2.1 Concretização do algoritmo para detetar interseções

De modo a corresponder ao enunciado solicitado e tendo os conceitos matemáticos abordados anteriormente como basilares foram elaboradas duas funções, a primeira de modo a determinar se existem interseções entre duas dadas arestas, se estas acontecem no interior do segmento de reta, e posteriormente apresentar as coordenadas de essa interseção. A segunda, percorre toda a lista de arestas do grafo, comparando cada aresta

com todas as outras e invoca a primeira função de modo a comprovar se existem interseções.

A função implementada para deteção de interseções entre arestas do grafo é baseada em conceitos matemáticos formais da geometria analítica e álgebra linear, sendo uma aplicação direta das equações paramétricas de segmentos de reta e da regra de Cramer para sistemas lineares 2×2.

No código, cada segmento de reta é definido por dois pontos extremos. A equação paramétrica é usada implicitamente ao calcular as diferenças entre as coordenadas dos pontos, o que define a direção do segmento. Estes cálculos correspondem às expressões matemáticas das equações paramétricas dos segmentos.

```
int dx1 = x2 - x1;
int dy1 = y2 - y1;
int dx2 = x4 - x3;
int dy2 = y4 - y3;
```

Figura 33 - Cálculos correspondentes às expressões matemáticas das equações paramétricas

O cálculo do determinante principal é determinado nestas linhas de código, tendo sido posteriormente verificado se este era igual a zero de modo a excluir os casos em que os segmentos de reta são paralelos ou colineares.

```
// Determinante do sistema
int det = dx1 * (-dy2) - dy1 * (-dx2);
if (det == 0) {
    // Segmentos paralelos (ou colineares)
    return false;
}
```

Figura 34 - Cálculo do determinante principal

O cálculo do Det(t) e Det(u) e posterior obtenção do valor dos parâmetros t e u, são comprovadas neste excerto de código:

```
double t = (double)(rx * (-dy2) - ry * (-dx2)) / det;
double u = (double)(dx1 * ry - dy1 * rx) / det;
```

Figura 35 - Cálculo de Det(t), Det(u), e dos parâmetros t e u

Seguidamente é verificado se os parâmetros se encontram entre zero e um de modo a comprovar que a interseção se verifica no interior dos segmentos de reta.

```c
if (t >= 0 && t <= 1 && u >= 0 && u <= 1) {
    if (ix) *ix = x1 + t * dx1;
    if (iy) *iy = y1 + t * dy1;
    return true;
}
```

Figura 36 - Verificação dos parâmetros e cálculo das interseções

Este cálculo corresponde à substituição de t na equação paramétrica do primeiro segmento, fornecendo o ponto de interseção exato.

Em suma, a implementação em C traduz diretamente a teoria matemática das interseções de segmentos de reta num algoritmo funcional e eficiente. Este algoritmo não só verifica a existência de interseções, como também calcula o ponto exato, sendo fundamental para a integridade topológica e análise de conflitos no grafo de antenas.

2.3 TRAVESSIA DE GRAFOS

No Projeto, implementaram-se dois algoritmos fundamentais de percurso em grafos: a DFS e a BFS. Ambas têm como objetivo visitar todos os vértices "acessíveis" a partir de um vértice inicial, mas utilizam estratégias diferentes.

A função dfs percorre o grafo de forma recursiva, explorando primeiro o caminho mais profundo antes de voltar atrás (Figura 36). Recebe como argumentos um vértice inicial, um vetor de marcação para saber os vértices que já foram visitados e as dimensões do grafo. A função começa por verificar se o vértice é válido e se já foi visitado; se não, marca-o como visitado, imprime as suas coordenadas e continua recursivamente a visitar os vértices adjacentes. Este método é ideal para explorar caminhos longos, detetar ciclos ou componentes conexos num grafo.

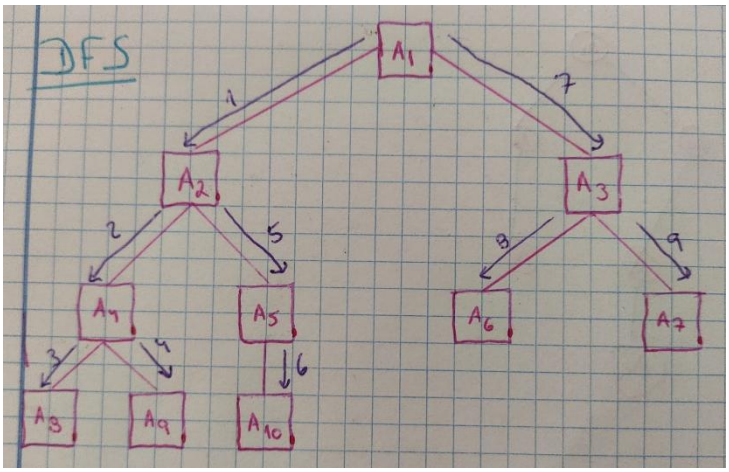


Figura 37 - DFS

Por outro lado, a função bfs utiliza uma fila para percorrer o grafo em largura (Figura 37). Começa no vértice inicial e visita todos os vizinhos diretos antes de passar para os vizinhos desses vizinhos. Para isso, cria uma fila dinâmica e um vetor de marcação semelhante à do DFS. A cada passo, remove o vértice da frente da fila, marca-o como visitado, imprime as suas coordenadas e adiciona todos os vértices adjacentes ainda não visitados à fila. Este método é particularmente eficaz para encontrar o caminho mais curto em grafos não ponderados e para garantir que todos os vértices a uma determinada distância são explorados antes de avançar.

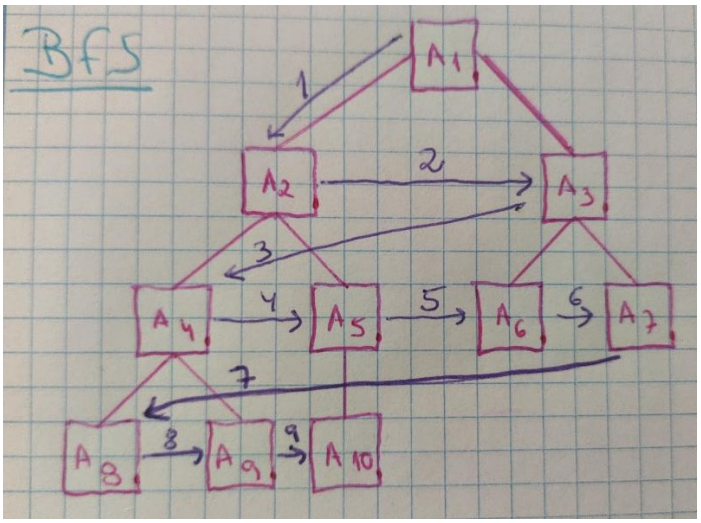


Figura 38 - BFS

Enquanto a DFS utiliza uma abordagem de pilha implícita (através da recursão), a BFS utiliza uma estrutura de fila explícita, o que diferencia o comportamento e a ordem de visita dos vértices. No contexto do Projeto, estas duas funções permitem analisar e explorar o grafo de antenas, proporcionando uma forma de testar a ligação e a estrutura das ligações entre antenas. Na Tabela 1, podemos ver as diferenças dos dois algoritmos:

Característica	Dfs	Bfs
Estratégia	Profundidade (recursão)	Largura (fila)
Ordem de visita	Vai fundo primeiro	Visita vizinhos primeiro
Tipo de estrutura	Pilha implícita (recursão)	Fila explícita
Utilidade	Detetar ciclos, caminhos	Encontrar menor caminho

Tabela 1 - dfs vs bfs

CONCLUSÃO

O desenvolvimento do Projeto Gestão de Antenas permitiu aplicar de forma prática os conhecimentos adquiridos UC de EDA, reforçando competências no manuseamento de listas ligadas, gestão de memória dinâmica e modularização de código em linguagem C. Ao longo do projeto, foi possível consolidar conceitos fundamentais para modularização lógica de ficheiros de cabeçalho e implementação, bem como o uso da ferramenta Doxygen para documentar o código-fonte de forma clara e estruturada.

A construção do programa revelou-se eficaz na leitura de antenas a partir de ficheiros de texto, no cálculo de interferências entre antenas com a mesma frequência e na sua representação gráfica sob a forma de matriz. Adicionalmente, o sistema provou ser robusto ao guardar corretamente os dados gerados num ficheiro binário, cumprindo todos os objetivos propostos. Os testes realizados demonstraram o funcionamento esperado da aplicação em diferentes cenários, desde matrizes simples a configurações mais complexas, validando assim a fiabilidade das funções implementadas.

Para além, no que concerne à Fase 2 do Projeto, a construção do grafo e a aplicação de algoritmos clássicos como DFS e BFS permitiram explorar a rede de antenas de forma estruturada, reforçando o domínio de conceitos associados à teoria dos grafos. As análises de interseções demonstram a utilidade e a flexibilidade da abordagem implementada. De realçar, ainda, que a estrutura modular adotada permitiu um desenvolvimento progressivo, facilitando a identificação e resolução de problemas. Deste modo, penso ter atingido todos os objetivos proposto no início deste Projeto.

Como principais dificuldades sentidas durante a elaboração deste projeto destaco a leitura de ficheiros a partir de matrizes de tamanho variável e conseguir definir um algoritmo capaz de solucionar o problema solicitado.

Apesar da complexidade envolvida em algumas fases o projeto contribuiu significativamente para o desenvolvimento de competências na área da programação estruturada e na produção de software bem documentado. Em suma, o projeto constituiu uma oportunidade enriquecedora de aprendizagem que serão fundamentais para o meu percurso académico e profissional.

Todo o código está disponível em: https://github.com/Filipe-Alexandre-Fernandes/Trabalho-EDA.git.

REFERÊNCIAS BIBLIOGRÁFICAS

Aulas EDA do Professor Luís Ferreira, disponíveis em: https://github.com/luferIPCA/EDA-LESI-2024-2025.git

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Damas, L. (2015). *Linguagem C* (24ªedição). FCA.

Franke, B. (2014). *Systems Programming in Unix/Linux*. Springer.

Kernighan, B. W., & Pike, R. (1999). *The Practice of Programming*. Addison-Wesley.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C Programming: A Modern Approach* (2nd ed.). W. W. Norton & Company.

Knuth, D. E. (1997). *The Art of Computer Programming: Volume 1 – Fundamental Algorithms* (3rd ed.). Addison-Wesley.

Kochan, S. G. (2014). *Programming in C* (4th ed.). Sams Publishing.

Love, R. (2010). *Linux System Programming* (2nd ed.). O'Reilly Media.

Ritchie, D. M. (1993). The development of the C language. *ACM SIGPLAN Notices*, 28(3), 201–208.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). *Operating System Concepts* (10th ed.). Wiley.

Tanenbaum, A. S. (2007). *Structured Computer Organization* (5th ed.). Pearson Education.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C* (2nd ed.). Pearson.

ANEXOS

ANEXO I – ENUNCIADO DO TRABALHO PRÁTICO (FASE 1)

Projeto de Avaliação

Estruturas de Dados Avançadas (EDA)

EST-IPCA

Barcelos

5 de março de 2025

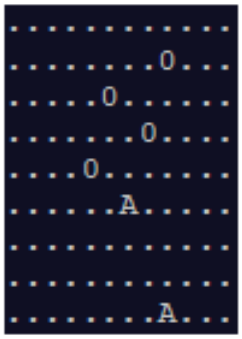
Motivação

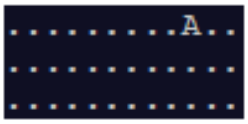
Este projeto de avaliação de realização individual da Unidade Curricular (UC) *Estruturas de Dados Avançadas* (EDA), integrada no 2° semestre do 1° ano, visa o reforço e a aplicação dos conhecimentos adquiridos ao longo do semestre.

Com este projeto de avaliação pretende-se sedimentar os conhecimentos relativos à definição e manipulação de estruturas de dados dinâmicas na linguagem de programação C. Este documento deve ser visto como uma referência para uma abordagem clássica de desenvolvimento de soluções de software para um problema de dimensão média. A implementação das soluções deverá considerar estruturas de dados dinâmicas, armazenamento em ficheiro, modularização e apresentar uma estruturação e documentação com Doxygen.

Contextualização

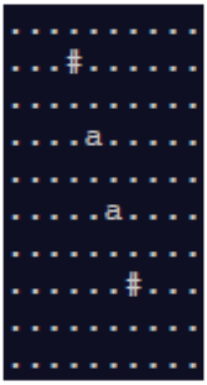
Pretende-se considerar uma cidade com várias antenas. Cada antena é sintonizada numa frequência específica indicada por um caracter. O mapa das antenas com as suas localizações (coordenadas na matriz) e frequências é representado através de uma matriz. Por exemplo:



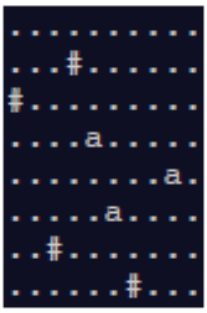


Considere que o sinal de cada antena aplica um efeito nefasto em localizações específicas L com base nas frequências de ressonância das antenas. Em particular, o efeito nefasto ocorre em qualquer localização L que esteja perfeitamente alinhada com duas antenas da mesma frequência - mas apenas quando uma das antenas está duas vezes mais distante que a outra. Isso significa que para qualquer par de antenas com a mesma frequência, existem duas localizações, uma de cada lado das antenas.

A título de exemplo, para duas antenas com frequência a, as localizações com efeito nefasto encontram-se representadas com #:



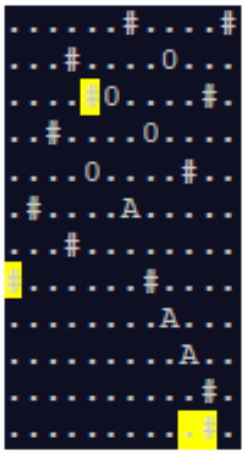
Adicionar uma terceira antena com a mesma frequência cria várias localizações adicionais com efeito nefasto:





Antenas com frequências diferentes (caracteres diferentes) não criam localizações com efeito nefasto. Localizações com efeito nefasto podem ocorrer em locais que contêm antenas. Uma localização com efeito nefasto pode surgir na sequência das várias combinações de antenas em simultâneo.

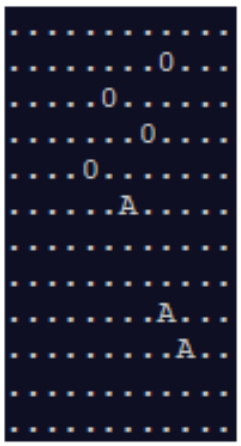
O primeiro exemplo tem antenas com duas frequências diferentes (A e O), dando origem as localizações com efeito nefasto seguintes:



Fase 1 - Listas ligadas

Considerando a contextualização supra-mencionada, procure implementar as funcionalidades seguintes:

1. Definição de uma estrutura de dados *ED*, para a representação das antenas, sob a forma de uma lista ligada simples. Cada registo da lista ligada deverá conter a frequência de ressonância de uma antena e suas coordenadas;
2. Carregamento para uma estrutura de dados *ED* dos dados das antenas constantes num ficheiro de texto. A operação deverá considerar matrizes de caracteres com qualquer dimensão. A título de exemplo, o ficheiro de texto deverá respeitar o formato seguinte:



3. Implementar operações de manipulação da lista ligada do tipo *ED*, incluindo:
    a. Inserção de uma nova antena na lista ligada;
    b. Remoção de uma antena constante na lista ligada;
    c. Dedução automática das localizações com efeito nefasto e respetiva representação sob a forma de uma lista ligada;
    d. Listagem de forma tabular na consola das antenas e localizações com efeito nefasto.

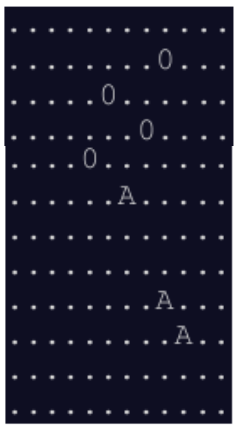
ANEXO II – ENUNCIADO DO TRABALHO PRÁTICO (FASE 2)

Fase 2 - Grafos

Na segunda fase, pretende-se aplicar conceitos avançados de teoria dos grafos e programação em C para resolver o mesmo problema computacional.

Assim, procure implementar as funcionalidades seguintes:

1. Definir um tipo de dados *GR* para representação de grafos, permitindo suportar um número qualquer de vértices. Cada vértice deverá representar uma antena e a sua localização numa cidade. As arestas deverão interligar somente antenas com frequências de ressonância iguais.
2. Carregamento dos dados referentes às antenas de uma cidade, constantes num ficheiro de texto, devendo dar origem a um grafo com tipo de dados *GR*. A título de exemplo, o ficheiro de texto deverá respeitar o formato seguinte:



3. Implementar operações de manipulação de grafos do tipo *GR*, incluindo:
    a. Procura em profundidade a partir de uma determinada antena, listando na consola as coordenadas das antenas alcançadas;
    b. Procura em largura a partir de uma determinada antena, listando na consola as coordenadas das antenas alcançadas;
    c. Identificar todos os caminhos possíveis entre duas determinadas antenas, listando na consola as sequência de arestas interligando as antenas;
    d. Dadas duas frequências de ressonância distintas *A* e *B*, listar na consola todas as intersecções de pares de antenas com ressonância *A* e *B*, indicando as respectivas coordenadas.

Documente extensivamente o código desenvolvido, explicando a lógica por trás das principais funções e decisões de implementação. Execute casos de teste com matrizes de diferentes dimensões e complexidades, demonstrando a eficiência da solução desenvolvida.